

MULTI-AGENT SYSTEM DESIGN

Agentic Pharma Insights

A master-and-worker agent architecture that compresses pharma portfolio research — trials, trade, patents, internal dossiers — from a months-long literature review into a cited brief.

• CONCEPT & ARCHITECTURE

• NOT A SHIPPED PRODUCT

buildbyhet.me • github.com/Het161

What EY actually asked for

A multinational generic pharmaceutical company with heavy US exposure wants out of the low-margin generics squeeze. Its route: **repurpose already-approved molecules** — new indications, alternative dosage forms, different patient populations.

2–3

MONTHS

The time a single opportunity-scouting literature review takes today, across multiple iterations. That is the number the system has to beat.

The ask was a **Master Agent** that connects to regulatory sites, trial registries, journals, subscription databases and the client's own documents; takes plain-language questions; and returns an archived, citable report.

Five sources, no shared key

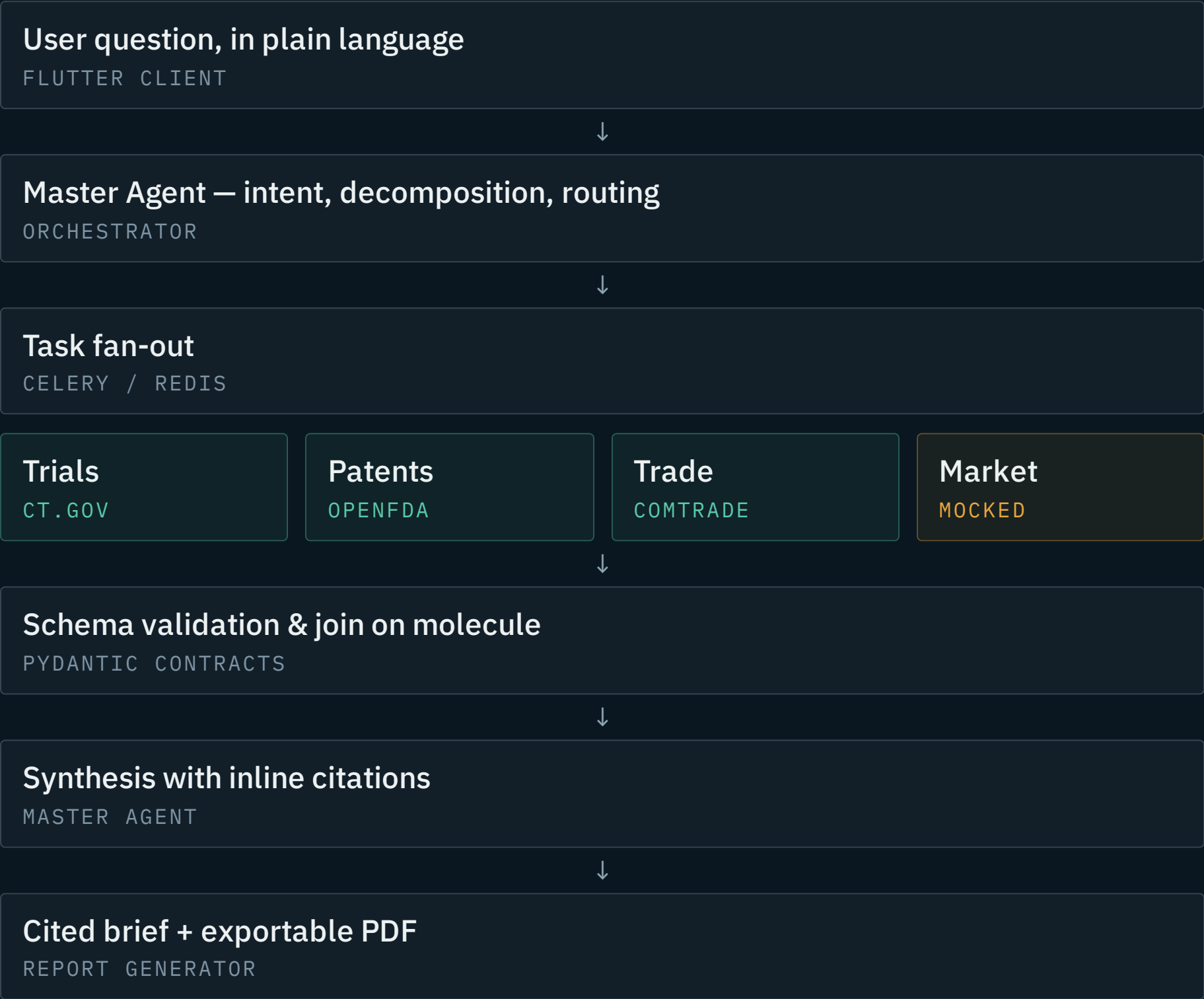
This is not a search problem. Each source answers a different question in a different vocabulary, and nothing joins them automatically.

MARKET	Sales and growth by therapy area — behind a paid licence
TRADE	Import/export flows keyed by HS commodity code, not by molecule
PATENTS	Filing and expiry dates keyed by application number, not by brand
TRIALS	Pipeline keyed by NCT ID, condition and mechanism of action
INTERNAL	Strategy decks and field reports — unstructured PDF prose

A strategist's real question — *"where is there unmet need we could actually reach?"* — needs four of these at once, joined on a molecule, and every claim traceable back to where it came from.

Master routes, workers fetch

One orchestrator owns intent, decomposition and synthesis. Workers are parallel and replaceable — each owns exactly one source.



Where every claim comes from

The brief permitted mock APIs throughout. We mocked only what is genuinely licence-gated, and wired the rest to real public endpoints.

Clinical trials

Pipeline by condition, phase, sponsor

ClinicalTrials.gov REST v2

● LIVE

Patent expiry

Listed patents, exclusivity, TE codes

openFDA – Orange Book

● LIVE

Patent landscape

Filings, assignees, CPC classes

USPTO Open Data Portal

● LIVE

EXIM trade

Bilateral API & formulation flows

UN Comtrade – HS 29 / 3003 / 3004

● LIVE

Market sizing

Mocked to a fixed schema

IQVIA – licence-gated

● MOCK

Internal docs

Synthetic corpus, RAG retrieval

Client PDFs → vector store

● MOCK

Web intelligence

Guidelines, publications, news

Allowlisted search + fetch

● LIVE

One decision worth calling out. For drug patent expiry, the Orange Book — not a general patent search — is the correct source: it lists exactly the patents and exclusivity periods that gate generic entry.

One question, end to end

USER PROMPT

"Which respiratory indications show low competition but high patient burden in India?"

01 Master decomposes

Splits into burden, pipeline density, and entry-window sub-questions. Asks a clarifying question if region or mechanism is ambiguous.

02 Workers run in parallel

Trials agent counts active studies by phase and sponsor. Trade agent pulls formulation flows. Patent agent checks expiry inside 24 months.

03 Validate, then join

Each worker returns a typed object or fails loudly. Results join on molecule; anything unresolved is reported as a gap, not smoothed over.

04 Synthesise with sources

Master composes a ranked shortlist. Every row carries the reference it came from; uncited assertions are dropped.

RETURNED

A ranked table of candidate indications, each with trial count, entry window and trade signal — plus a one-click PDF that keeps the citations intact.

The rules that make it pharma

Retrieval alone produces a reading list. These heuristics are what turn joined data into something a portfolio strategist would actually act on — and they run as explicit code, not as instructions buried in a prompt.

High disease burden + low active trial count

→ Whitespace — worth a deeper look

Listed patent expiring within 24 months

→ Generic or biosimilar entry window opening

Approved molecule with trials in a second indication

→ Repurposing candidate — the client's actual goal

Rising import dependence on a single origin country

→ Supply concentration risk on the API

Guardrails

A wrong number in a portfolio brief is worse than no brief. Most of the design effort went here.

Citation or nothing

Every table cell and claim carries its source reference. An assertion that cannot be traced is dropped, never hedged into vagueness.

Typed worker contracts

Each worker returns a validated schema. Invalid output retries once, then surfaces as an explicit gap in the brief.

Deterministic runs

Temperature zero and a fixed tool order, so the same question twice gives the same brief — a hard requirement for anything a committee will cite.

Human before export

The analyst edits and approves the draft before a PDF exists. The system proposes; a person signs.

Visible data mode

A live/mock switch is shown in the interface, so a reader always knows whether a number is real.

What we chose, and one regret

BACKEND	Django REST — Python agents, JWT auth
ASYNC	Celery + Redis — worker fan-out, caching, run leases
STORE	PostgreSQL for runs and metadata; vector store for document RAG
CLIENT	Flutter — one codebase for web and mobile chat
DELIVERY	Docker, GitHub Actions CI/CD, staging environment, managed secrets

● IN HINDSIGHT

The brief pointed at **LangGraph or CrewAI**; we hand-rolled orchestration on Celery. That bought precise control of retries and observability, but gave up graph-native state and checkpointing we then had to reinvent. The better split: LangGraph for the master's reasoning loop, Celery for the worker fan-out.

What this is, and isn't

● DESIGNED

System architecture · agent contracts and boundaries · data-source selection and justification · decision heuristics · guardrail model · conversation and report flow

● NOT BUILT

No running system, no demo video, no benchmark numbers. This was a concept and design submission — presenting it as a shipped product would be a lie an interviewer would catch in one question.

Three things I'd do differently

- 01 Build one vertical slice — trials to cited PDF — before designing seven agents on paper. A thin working path beats a wide diagram.
- 02 Commit to the Orange Book for expiry from day one. Generic patent search was the wrong instrument for the question being asked.
- 03 Ship the live/mock toggle first. Demo credibility collapses the moment a judge cannot tell which numbers are real.

The part worth keeping

The interesting problem here was never the agents. It was **provenance** — making a generated answer carry its sources all the way to the export, and designing the system so an untraceable claim has nowhere to hide.

That constraint is portable. It applies to any system where someone has to defend the output in a room full of people who will check.

- LIVE PUBLIC API
- LICENCE-GATED, MOCKED
- MODEL-DERIVED

Het · [your role — edit before posting]

buildbyhet.me · github.com/Het161 · het@buildbyhet.me